

```
In [1]: %pip install earthengine-api pandas matplotlib numpy scipy
```

Requirement already satisfied: earthengine-api in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.6.14)

Requirement already satisfied: pandas in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (2.3.3)

Requirement already satisfied: matplotlib in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (3.10.1)

Requirement already satisfied: numpy in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (2.2.5)

Requirement already satisfied: scipy in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.15.2)

Requirement already satisfied: google-cloud-storage in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (3.4.1)

Requirement already satisfied: google-api-python-client>=1.12.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (2.186.0)

Requirement already satisfied: google-auth>=1.4.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (2.42.1)

Requirement already satisfied: google-auth-http2lib>=0.0.3 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (0.2.1)

Requirement already satisfied: http2lib2<1dev,>=0.9.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (0.31.0)

Requirement already satisfied: requests in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from earthengine-api) (2.32.3)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.2)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.3.2)

Requirement already satisfied: cycler>=0.10 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (4.57.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.4.8)

Requirement already satisfied: packaging>=20.0 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (24.2)

Requirement already satisfied: pillow>=8 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (11.2.1)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (3.2.3)

Requirement already satisfied: google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-api-python-client>=1.12.1->earthengine-api) (2.28.1)

Requirement already satisfied: uritemplate<5,>=3.0.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-api-python-client>=1.12.1->earthengine-api) (4.2.0)

Requirement already satisfied: cachetools<7.0,>=2.0.0 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-auth>=1.4.1->earthengine-api) (6.2.1)

Requirement already satisfied: pyasn1-modules>=0.2.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-auth>=1.4.1->earthengine-api) (0.4.2)

Requirement already satisfied: rsa<5,>=3.1.4 in c:\users\admin\appdata\local\program

s\python\python313\lib\site-packages (from google-auth>=1.4.1->earthengine-api) (4.9.1)
Requirement already satisfied: six>=1.5 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-cloud-storage->earthengine-api) (2.5.0)
Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-cloud-storage->earthengine-api) (2.7.2)
Requirement already satisfied: google-crc32c<2.0.0,>=1.1.3 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-cloud-storage->earthengine-api) (1.7.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from requests->earthengine-api) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from requests->earthengine-api) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from requests->earthengine-api) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from requests->earthengine-api) (2025.4.26)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5->google-api-python-client>=1.12.1->earthengine-api) (1.71.0)
Requirement already satisfied: protobuf!=3.20.0,!3.20.1,!4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<7.0.0,>=3.19.5 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5->google-api-python-client>=1.12.1->earthengine-api) (6.33.0)
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5->google-api-python-client>=1.12.1->earthengine-api) (1.26.1)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth>=1.4.1->earthengine-api) (0.6.1)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.3.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [2]: import ee
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
```

```
In [3]: # Initialize Earth Engine

try:
    ee.Initialize()
    print(" Earth Engine initialized successfully\n")
except:
    print("Authenticating Earth Engine...")
```

```
ee.Authenticate()
ee.Initialize()
print(" Earth Engine initialized successfully\n")
```

Earth Engine initialized successfully

In [4]: *# Lake Selection - I chose Lake Tahoe in Nevada*

```
# Lake Tahoe center coordinates
lon = -120.0324
lat = 39.0968

# STEP 2: Create Region of Interest (ROI)

print("Creating Region of Interest (ROI)...")
point = ee.Geometry.Point([lon, lat])
buffered = point.buffer(200) # 200m radius
roi = buffered.bounds()

print(f" ROI created: 200m buffer (400m diameter)")
print(f" Center: {lat}°N, {lon}°W\n")
```

Creating Region of Interest (ROI)...
ROI created: 200m buffer (400m diameter)
Center: 39.0968°N, -120.0324°W

In [5]: *#Cloud/Snow Masking Functions*

```
def mask_landsat457_sr(image):
    """Cloud and snow masking for Landsat 4, 5, 7"""
    qa = image.select('QA_PIXEL')

    # Bit masks
    cloud_bit = 1 << 3
    cloud_shadow_bit = 1 << 4
    snow_bit = 1 << 5
    cirrus_bit = 1 << 2

    # Cloud confidence (bits 8-9)
    cloud_confidence = qa.rightShift(8).bitwiseAnd(3)

    # Create mask
    mask = qa.bitwiseAnd(cloud_bit).eq(0) \
        .And(qa.bitwiseAnd(cloud_shadow_bit).eq(0)) \
        .And(qa.bitwiseAnd(snow_bit).eq(0)) \
        .And(qa.bitwiseAnd(cirrus_bit).eq(0)) \
        .And(cloud_confidence.lt(2))

    return image.updateMask(mask)

def mask_landsat89_sr(image):
    """Cloud and snow masking for Landsat 8, 9"""
    qa = image.select('QA_PIXEL')

    # Bit masks
```

```

cloud_bit = 1 << 3
cloud_shadow_bit = 1 << 4
snow_bit = 1 << 5
cirrus_bit = 1 << 2

# Cloud confidence
cloud_confidence = qa.rightShift(8).bitwiseAnd(3)

# Create mask
mask = qa.bitwiseAnd(cloud_bit).eq(0) \
        .And(qa.bitwiseAnd(cloud_shadow_bit).eq(0)) \
        .And(qa.bitwiseAnd(snow_bit).eq(0)) \
        .And(qa.bitwiseAnd(cirrus_bit).eq(0)) \
        .And(cloud_confidence.lt(2))

return image.updateMask(mask)

def apply_scale_offset_L457(image):
    """Apply scale and offset for Landsat 4, 5, 7"""
    thermal = image.select('ST_B6') \
        .multiply(0.00341802) \
        .add(149.0) \
        .rename('temperature_K')
    return image.addBands(thermal)

def apply_scale_offset_L89(image):
    """Apply scale and offset for Landsat 8, 9"""
    thermal = image.select('ST_B10') \
        .multiply(0.00341802) \
        .add(149.0) \
        .rename('temperature_K')
    return image.addBands(thermal)

```

```

In [6]: # Temperature Calculation
def add_temperature_stats(image):
    """
    Calculate mean temperature over ROI and add as image properties
    Handles null values properly
    """
    # Get the masked temperature band
    temp_band = image.select('temperature_K')

    # Calculate statistics
    stats = temp_band.reduceRegion(
        reducer=ee.Reducer.mean().combine(
            reducer2=ee.Reducer.count(),
            sharedInputs=True
        ),
        geometry=roi,
        scale=30,
        maxPixels=1e9
    )

    mean_temp = stats.get('temperature_K_mean')
    pixel_count = stats.get('temperature_K_count')

```

```

# Calculate Celsius only if mean_temp is not null
mean_temp_c = ee.Algorithms.If(
    ee.Algorithms.IsEqual(mean_temp, None),
    None,
    ee.Number(mean_temp).subtract(273.15)
)

# Add properties to image (server-side)
return image.set({
    'mean_temp_K': mean_temp,
    'mean_temp_C': mean_temp_c,
    'valid_pixels': pixel_count,
    'date_string': image.date().format('YYYY-MM-dd'),
    'timestamp': image.date().millis()
})

```

In [7]: *#Process Each Landsat Mission*

```

print("Processing Landsat Collections...")
print("-" * 70)

all_temperature_data = []

# Define satellite collections
satellites = {
    'Landsat 4': {
        'collection': 'LANDSAT/LT04/C02/T1_L2',
        'mask_func': mask_landsat457_sr,
        'scale_func': apply_scale_offset_L457,
        'years': '1982-1993'
    },
    'Landsat 5': {
        'collection': 'LANDSAT/LT05/C02/T1_L2',
        'mask_func': mask_landsat457_sr,
        'scale_func': apply_scale_offset_L457,
        'years': '1984-2012'
    },
    'Landsat 7': {
        'collection': 'LANDSAT/LE07/C02/T1_L2',
        'mask_func': mask_landsat457_sr,
        'scale_func': apply_scale_offset_L457,
        'years': '1999-present'
    },
    'Landsat 8': {
        'collection': 'LANDSAT/LC08/C02/T1_L2',
        'mask_func': mask_landsat89_sr,
        'scale_func': apply_scale_offset_L89,
        'years': '2013-present'
    },
    'Landsat 9': {
        'collection': 'LANDSAT/LC09/C02/T1_L2',
        'mask_func': mask_landsat89_sr,
        'scale_func': apply_scale_offset_L89,
        'years': '2021-present'
    }
}

```

```

}

# Process each satellite
for sat_name, sat_info in satellites.items():
    print(f"\n{sat_name} ({sat_info['years']}):")
    print(f"  Loading collection...")

    # Load collection
    collection = ee.ImageCollection(sat_info['collection']) \
        .filterBounds(roi)

    total_images = collection.size().getInfo()
    print(f"  Total images available: {total_images}")

    if total_images == 0:
        print(f"    No images found")
        continue

    # Apply cloud/snow masking and temperature conversion (SERVER-SIDE)
    print(f"  Applying masks and calculating temperatures...")
    processed = collection \
        .map(sat_info['mask_func']) \
        .map(sat_info['scale_func']) \
        .map(add_temperature_stats)

    # Filter out completely cloudy images (where mean_temp_K is null)
    valid_images = processed.filter(ee.Filter.notNull(['mean_temp_K']))

    print(f"  Counting valid images...")
    valid_count = valid_images.size().getInfo()
    cloudy_count = total_images - valid_count

    print(f"    Valid images: {valid_count}")
    print(f"    Cloudy/masked images: {cloudy_count}")

    # Extract properties to list
    if valid_count > 0:
        print(f"    Downloading results...")

        def extract_properties(image):
            return ee.Feature(None, {
                'satellite': sat_name,
                'date': image.get('date_string'),
                'timestamp': image.get('timestamp'),
                'mean_temp_K': image.get('mean_temp_K'),
                'mean_temp_C': image.get('mean_temp_C'),
                'valid_pixels': image.get('valid_pixels')
            })

        features = valid_images.map(extract_properties)
        feature_collection = ee.FeatureCollection(features)

    try:
        results = feature_collection.getInfo()

```

```

        # Extract data from results
        for feature in results['features']:
            props = feature['properties']
            if props['mean_temp_K'] is not None:
                all_temperature_data.append(props)

        print(f"    Downloaded {len(results['features'])} measurements")
    except Exception as e:
        print(f"    Error downloading: {str(e)}")
        continue

print("\n" + "=" * 70)
print(f"TOTAL VALID MEASUREMENTS: {len(all_temperature_data)}")
print("=" * 70 + "\n")

# Export to CSV
csv_filename = 'lake_tahoe_temperature_data.csv'
print(f"Exporting data to {csv_filename}...")

# Create DataFrame
df = pd.DataFrame(all_temperature_data)
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date')

# Save to CSV
df.to_csv(csv_filename, index=False)
print(f"Data exported successfully\n")

# Display sample data
print("Sample of collected data:")
print(df.head(10).to_string(index=False))
print(f"\n... {len(df)} total records ...\n")

```


Processing Landsat Collections...

Landsat 4 (1982-1993):

Loading collection...
Total images available: 2
Applying masks and calculating temperatures...
Counting valid images...
Valid images: 1
Cloudy/masked images: 1
Downloading results...
Downloaded 1 measurements

Landsat 5 (1984-2012):

Loading collection...
Total images available: 508
Applying masks and calculating temperatures...
Counting valid images...
Valid images: 384
Cloudy/masked images: 124
Downloading results...
Downloaded 384 measurements

Landsat 7 (1999-present):

Loading collection...
Total images available: 525
Applying masks and calculating temperatures...
Counting valid images...
Valid images: 405
Cloudy/masked images: 120
Downloading results...
Downloaded 405 measurements

Landsat 8 (2013-present):

Loading collection...
Total images available: 276
Applying masks and calculating temperatures...
Counting valid images...
Valid images: 196
Cloudy/masked images: 80
Downloading results...
Downloaded 196 measurements

Landsat 9 (2021-present):

Loading collection...
Total images available: 89
Applying masks and calculating temperatures...
Counting valid images...
Valid images: 64
Cloudy/masked images: 25
Downloading results...
Downloaded 64 measurements

=====

TOTAL VALID MEASUREMENTS: 1050

=====

Exporting data to lake_tahoe_temperature_data.csv...

Data exported successfully

Sample of collected data:

date	mean_temp_C	mean_temp_K	satellite	timestamp	valid_pixels
1983-01-09	4.305210	277.455210	Landsat 4	410983703716	196
1984-05-27	10.226621	283.376621	Landsat 5	454529246818	196
1984-06-28	16.762523	289.912523	Landsat 5	457294078909	196
1984-09-16	17.715214	290.865214	Landsat 5	464206176564	196
1984-10-02	14.400891	287.550891	Landsat 5	465588576776	196
1985-01-22	4.438063	277.588063	Landsat 5	475265392568	196
1985-05-14	8.152273	281.302273	Landsat 5	484942176622	196
1985-06-15	16.453955	289.603955	Landsat 5	487706975558	196
1985-07-17	18.101586	291.251586	Landsat 5	490471764887	196
1985-08-02	16.551114	289.701114	Landsat 5	491854157065	196

... 1050 total records ...

```
In [8]: # STEP 7: Statistical Analysis
print("=" * 70)
print("STATISTICAL ANALYSIS")
print("=" * 70)

print("\nTemperature Statistics:")
print(df[['mean_temp_K', 'mean_temp_C']].describe())

print("\nData by Satellite:")
print(df.groupby('satellite').agg({
    'mean_temp_C': ['count', 'mean', 'std', 'min', 'max']
}).round(2))

# Calculate annual means
df['year'] = df['date'].dt.year
annual_means = df.groupby('year')['mean_temp_C'].mean()

# Calculate trend
years = annual_means.index.values
temps = annual_means.values

#Calculating p-value to test significance
slope, intercept, r_value, p_value, std_err = stats.linregress(years, temps)

print(f"\n" + "=" * 70)
print("TREND ANALYSIS")
print("=" * 70)
print(f"Temperature trend: {slope:.4f} °C/year")
print(f"Total change ({years[0]}-{years[-1]}): {slope * (years[-1]-years[0]):.2f} °C")
print(f"P-value: {p_value:.6f}")

if p_value < 0.05:
    print(f"\n STATISTICALLY SIGNIFICANT warming trend detected!")
else:
    print(f"\n Trend is not statistically significant (p > 0.05)")
```

```
=====
STATISTICAL ANALYSIS
=====
```

Temperature Statistics:

	mean_temp_K	mean_temp_C
count	1050.000000	1050.000000
mean	284.893489	11.743489
std	6.407156	6.407156
min	249.840334	-23.309666
25%	279.274919	6.124919
50%	285.147118	11.997118
75%	290.598723	17.448723
max	296.258116	23.108116

Data by Satellite:

		mean_temp_C			
	count	mean	std	min	max
satellite					
Landsat 4	1	4.31	NaN	4.31	4.31
Landsat 5	384	11.14	6.37	-23.31	21.87
Landsat 7	405	11.68	6.33	-6.87	22.00
Landsat 8	196	13.03	6.54	-5.05	23.11
Landsat 9	64	11.92	6.28	-1.51	22.33

```
=====
TREND ANALYSIS
=====
```

Temperature trend: 0.0585 °C/year
Total change (1983-2025): 2.46 °C
P-value: 0.005011

STATISTICALLY SIGNIFICANT warming trend detected!

```
In [9]: #Visualization
print(f"\nGenerating visualizations...")

fig = plt.figure(figsize=(16, 12))

# Plot 1: Complete time series with all satellites
ax1 = plt.subplot(3, 1, 1)
colors = {'Landsat 4': '#1f77b4', 'Landsat 5': '#ff7f0e',
          'Landsat 7': '#2ca02c', 'Landsat 8': '#d62728',
          'Landsat 9': '#9467bd'}

for satellite in df['satellite'].unique():
    sat_data = df[df['satellite'] == satellite]
    ax1.scatter(sat_data['date'], sat_data['mean_temp_C'],
               label=satellite, alpha=0.6, s=15, color=colors.get(satellite, 'gray'))

ax1.set_xlabel('Date', fontsize=12, fontweight='bold')
ax1.set_ylabel('Surface Temperature (°C)', fontsize=12, fontweight='bold')
ax1.set_title('Lake Tahoe Surface Temperature Time Series (1982-2025)\nAll Landsat',
              fontsize=14, fontweight='bold')
ax1.legend(loc='best', framealpha=0.9)
ax1.grid(True, alpha=0.3, linestyle='--')
```

```

# Plot 2: Annual mean temperature with trend line
ax2 = plt.subplot(3, 1, 2)
ax2.plot(annual_means.index, annual_means.values, 'o-',
        linewidth=2, markersize=8, alpha=0.7, label='Annual Mean')

# Add trend line
trend_line = slope * years + intercept
ax2.plot(years, trend_line, 'r--', linewidth=2,
        label=f'Trend: {slope:.4f}°C/year (R²={r_value**2:.3f})')

ax2.set_xlabel('Year', fontsize=12, fontweight='bold')
ax2.set_ylabel('Mean Temperature (°C)', fontsize=12, fontweight='bold')
ax2.set_title('Annual Mean Surface Temperature with Linear Trend',
        fontsize=14, fontweight='bold')
ax2.legend(loc='best', framealpha=0.9)
ax2.grid(True, alpha=0.3, linestyle='--')

# Plot 3: Seasonal patterns (monthly boxplot)
ax3 = plt.subplot(3, 1, 3)
df['month'] = df['date'].dt.month
monthly_data = [df[df['month'] == m]['mean_temp_C'].values for m in range(1, 13)]
bp = ax3.boxplot(monthly_data, labels=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
        patch_artist=True)

for patch in bp['boxes']:
    patch.set_facecolor('lightblue')
    patch.set_alpha(0.7)

ax3.set_xlabel('Month', fontsize=12, fontweight='bold')
ax3.set_ylabel('Surface Temperature (°C)', fontsize=12, fontweight='bold')
ax3.set_title('Seasonal Temperature Patterns (All Years Combined)',
        fontsize=14, fontweight='bold')
ax3.grid(True, alpha=0.3, linestyle='--', axis='y')

plt.tight_layout()
plt.savefig('lake_tahoe_temperature_analysis.png', dpi=300, bbox_inches='tight')
print(f" Plot saved as: lake_tahoe_temperature_analysis.png\n")

print("=" * 70)
print(f"\nOutput files created:")
print(f"  1. {csv_filename}")
print(f"  2. lake_tahoe_temperature_analysis.png")

plt.show()

```

Generating visualizations...

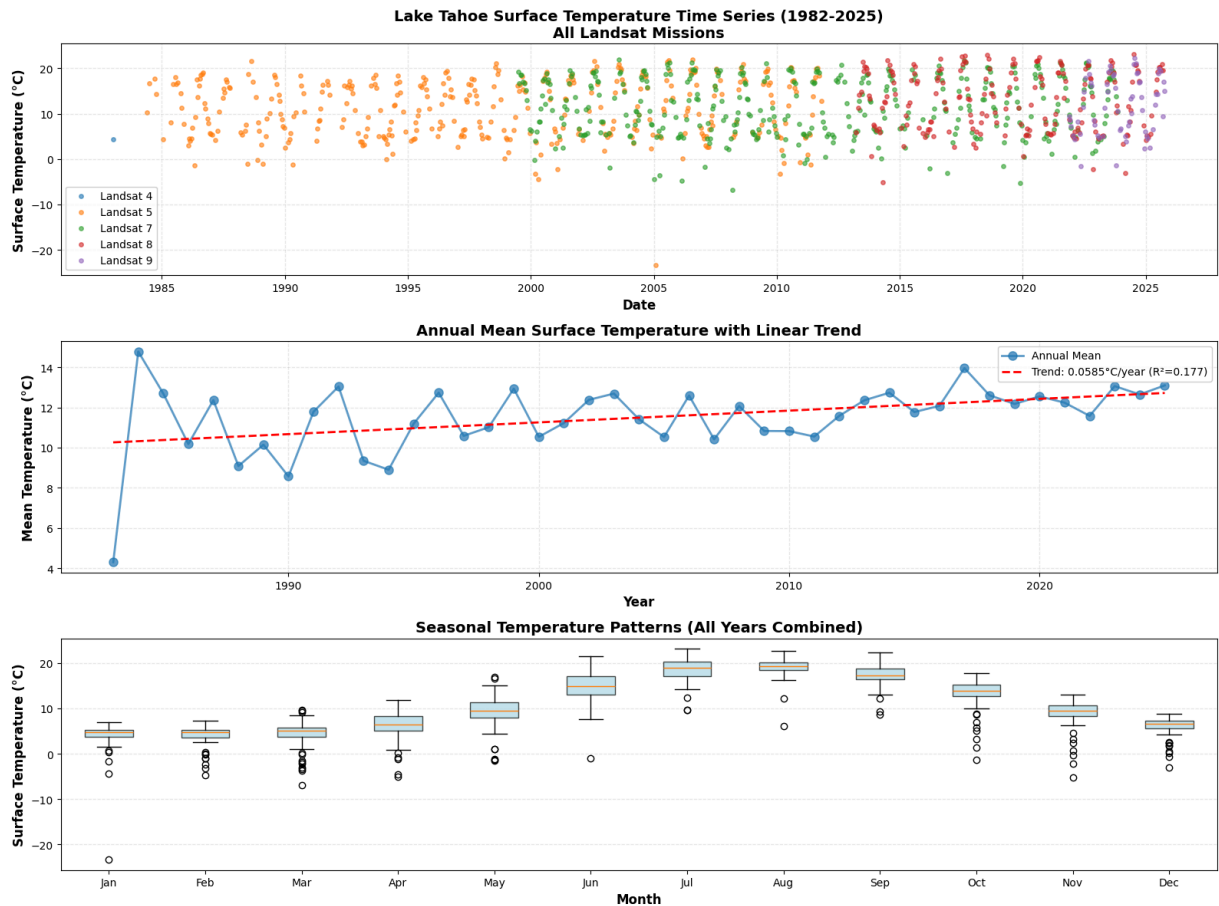
C:\Users\Admin\AppData\Local\Temp\ipykernel_2844\2070235878.py:45: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
bp = ax3.boxplot(monthly_data, labels=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
```

Plot saved as: lake_tahoe_temperature_analysis.png

Output files created:

1. lake_tahoe_temperature_data.csv
2. lake_tahoe_temperature_analysis.png



Findings: There is evidence of warming at Lake Tahoe in the graph. Over the roughly 42-year period depicted (mid-1980s to late 2020s), the linear trend shows a temperature increase of 0.0585°C annually, or about 2.5°C of warming. This is significant and consistent with more general patterns of global warming. The upward-sloping red trendline makes it evident that recent years have consistently seen higher temperatures than previous decades, with the most recent measurements clustering around $12\text{--}14^{\circ}\text{C}$ compared to $9\text{--}11^{\circ}\text{C}$ in the late 1980s and 1990s. The blue data points show significant year-to-year variability. The warming signal is statistically significant and alarming for the ecosystem of Lake Tahoe, even though the R^2 value of 0.177 shows that only roughly 18% of the temperature variance can be explained by the linear trend alone. The initial uptick in the graph was concerning for me. Upon research I found that since Lake Tahoe's water temperature would never actually fluctuate by 10°C in a single year, the first spike, which occurs around 1983–1984, when the temperature jumps from about 4.3°C to 14.8°C , is most likely a data artifact rather than a true climate signal. During the early days of Landsat 4's Thematic Mapper sensor's operation, a number of factors likely contributed to this anomaly: early calibration issues, nascent atmospheric correction algorithms that failed to adequately account for water vapor and other

atmospheric effects, possible cloud contamination in mixed pixels, and a lack of clear-sky observations that allowed inaccurate measurements to disproportionately affect the annual mean. This is supported by the fact that temperatures soon return to more stable levels in the 9–13°C range.